

Donation System Documentation

Overview

This document provides comprehensive information about the Workers Rights Watch donation system, including implementation details, payment gateway integrations, and setup instructions for Stripe, PayPal, and M-Pesa.

Table of Contents

1. [Features](#)
2. [Payment Gateway Integrations](#)
3. [Setup Instructions](#)
4. [API Routes](#)
5. [Environment Variables](#)
6. [Testing](#)
7. [Troubleshooting](#)

Features

🌟 Key Features

- **Multi-Currency Support:** Supports 10+ currencies with automatic conversion
- **Multiple Payment Methods:**
 - Stripe (Credit/Debit Cards)
 - PayPal
 - M-Pesa (Mobile Money - Kenya only)
- **Smart Currency Detection:** Automatically detects user's location and suggests appropriate currency
- **Real-time Amount Conversion:** Accurate currency conversion with proper formatting
- **Responsive Design:** Beautiful, modern UI that works on all devices
- **Test Mode Indicator:** Clear indication when in testing environment
- **Payment Status Tracking:** Real-time status updates for M-Pesa payments

🎨 UI Improvements

- **Dynamic Animated Background:** Beautiful gradient background with animated blobs
- **Modern Card Design:** Clean, professional card-based layout
- **Payment Method Cards:** Visual payment method selection with icons
- **Improved Typography:** Better readability and visual hierarchy
- **Smooth Animations:** Framer Motion animations for better UX
- **Responsive Layout:** Optimized for mobile, tablet, and desktop

Payment Gateway Integrations

1. Stripe Integration

How it works:

- User selects amount and currency
- User provides name and email
- System creates a Stripe Checkout Session
- User is redirected to Stripe's secure payment page
- After payment, user is redirected back with success status
- Webhook handles payment confirmation

Features:

- Secure hosted checkout page
- Supports all major credit/debit cards
- Automatic currency conversion to USD for processing
- Webhook integration for payment confirmation

API Routes:

- `POST /api/create-checkout-session` - Creates Stripe checkout session
- `POST /api/stripe-webhook` - Handles Stripe webhook events

2. PayPal Integration

How it works:

- User selects amount and currency
- User provides name and email
- System creates a PayPal order

- User is redirected to PayPal's payment page
- After payment approval, user is redirected back
- System captures the payment automatically

Features:

- Secure PayPal checkout
- Supports PayPal balance and credit/debit cards
- Automatic order capture
- Sandbox and production modes

API Routes:

- POST /api/paypal-create-order - Creates PayPal order
- POST /api/paypal-capture-order - Captures PayPal payment

3. M-Pesa Integration

How it works:

- User selects amount (in KES)
- User provides M-Pesa phone number only
- System initiates STK Push to user's phone
- User enters M-Pesa PIN on their phone
- System polls for payment status
- Payment is confirmed via callback

Features:

- Phone number only (no name/email required)
- Real-time payment status polling
- Automatic callback handling
- Supports both sandbox and production

API Routes:

- POST /api/mpesa-payment - Initiates M-Pesa STK Push
- POST /api/mpesa-callback - Handles M-Pesa payment callbacks
- GET /api/mpesa-status - Checks payment status

Setup Instructions

Getting Stripe Account

1. Sign Up for Stripe

- Visit: <https://stripe.com>
- Click "Sign up" and create an account
- Complete business verification (required for live mode)

2. Get API Keys

- Log in to Stripe Dashboard: <https://dashboard.stripe.com>
- Navigate to **Developers** → **API keys**
- Copy your **Publishable key** (starts with `pk_test_` or `pk_live_`)
- Copy your **Secret key** (starts with `sk_test_` or `sk_live_`)
- ⚠ Keep your secret key secure and never expose it publicly

3. Set Up Webhook

- In Stripe Dashboard, go to **Developers** → **Webhooks**
- Click **Add endpoint**
- Enter your webhook URL: `https://yourdomain.com/api/stripe-webhook`
- Select events to listen to:
 - `checkout.session.completed`
 - `payment_intent.succeeded`
 - `payment_intent.payment_failed`
- Copy the **Webhook signing secret** (starts with `whsec_`)

4. Test Mode

- Use test keys for development
- Test card: 4242 4242 4242 4242
- Any future expiry date (e.g., 12/34)
- Any 3-digit CVC
- Any ZIP code

Getting PayPal Account

1. Sign Up for PayPal Business Account

- Visit: <https://www.paypal.com/business>
- Click "Sign Up" and create a business account
- Complete business verification

2. Get Developer Credentials

- Visit PayPal Developer Dashboard: <https://developer.paypal.com>
- Log in with your PayPal account
- Navigate to **Dashboard → My Apps & Credentials**
- Click **Create App**
- Choose **Sandbox** for testing or **Live** for production
- Name your app (e.g., "Workers Rights Watch Donations")
- Copy your **Client ID** and **Client Secret**

3. Sandbox Testing

- PayPal provides sandbox accounts for testing
- Go to **Dashboard → Sandbox → Accounts**
- Use the provided test buyer and seller accounts
- Or create custom test accounts

4. Production Setup

- Switch to **Live** mode in Developer Dashboard
- Get your live **Client ID** and **Client Secret**
- Update your environment variables

Getting M-Pesa (Daraja API) Account

1. Register on Safaricom Developer Portal

- Visit: <https://developer.safaricom.co.ke>
- Click "Get Started" or "Sign Up"
- Create an account with your email

2. Create an App

- Log in to the Developer Portal
- Navigate to **My Apps**
- Click **Create App**
- Fill in app details:

- App Name: "Workers Rights Watch Donations"
- Description: "Donation payment system"
- Environment: Choose **Sandbox** for testing
- Select required APIs:
 - M-Pesa Express (STK Push)
 - M-Pesa Query

3. Get Credentials

- After creating the app, you'll get:
 - **Consumer Key**
 - **Consumer Secret**
- Copy these credentials

4. Get Shortcode and Passkey

- For **Sandbox**:
 - Shortcode: 174379 (test shortcode)
 - Passkey: Provided in the Developer Portal under your app
- For **Production**:
 - You need to apply for a Paybill or Till number
 - Contact Safaricom for production credentials
 - Passkey is provided after approval

5. Configure Callback URL

- In your app settings, set the callback URL:
 - `https://yourdomain.com/api/mpesa-callback`
- This URL must be publicly accessible (not localhost)

6. Test Credentials

- Sandbox test phone: Use any Safaricom number format
- Test with Safaricom's test numbers if provided

API Routes

Stripe Routes

POST /api/create-checkout-session

Creates a Stripe Checkout Session.

Request Body:

```
{
  "amount": 25.00,
  "currency": "usd",
  "displayAmount": 3238,
  "displayCurrency": "KES",
  "name": "John Doe",
  "email": "john@example.com"
}
```

Response:

```
{
  "success": true,
  "sessionId": "cs_test_...",
  "url": "https://checkout.stripe.com/..."
}
```

POST /api/stripe-webhook

Handles Stripe webhook events. Must be configured in Stripe Dashboard.

Headers:

- stripe-signature : Stripe webhook signature

Events Handled:

- checkout.session.completed
- payment_intent.succeeded
- payment_intent.payment_failed

PayPal Routes

POST /api/paypal-create-order

Creates a PayPal order.

Request Body:

```
{
  "amount": 25.00,
  "currency": "USD",
  "name": "John Doe",
  "email": "john@example.com"
}
```

Response:

```
{
  "success": true,
  "orderID": "50190127TN364715T",
  "approvalUrl": "https://www.sandbox.paypal.com/checkoutnow?..."
}
```

POST /api/paypal-capture-order

Captures a PayPal order after user approval.

Request Body:

```
{
  "orderID": "50190127TN364715T"
}
```

Response:

```
{
  "success": true,
  "orderID": "50190127TN364715T",
  "status": "COMPLETED",
  "payer": { ... }
}
```

M-Pesa Routes

POST /api/mpesa-payment

Initiates M-Pesa STK Push.

Request Body:


```
{
  "amount": 3238,
  "phone": "254712345678",
  "name": "M-Pesa Donor",
  "email": "donor@mpesa.local"
}
```

Note: name and email are placeholders for M-Pesa. Only amount and phone are required.

Response:

```
{
  "success": true,
  "checkoutRequestID": "ws_CO_191220231020440123456789",
  "customerMessage": "Success. Request accepted for processing"
}
```

POST /api/mpesa-callback

Handles M-Pesa payment callbacks. Called by Safaricom when payment is processed.

Request Body: (from Safaricom)

```
{
  "Body": {
    "stkCallback": {
      "CheckoutRequestID": "...",
      "ResultCode": 0,
      "ResultDesc": "The service request is processed successfully.",
      "CallbackMetadata": { ... }
    }
  }
}
```

GET /api/mpesa-status

Checks the status of an M-Pesa payment.

Query Parameters:

- checkoutRequestID : The checkout request ID from payment initiation

Response:

```
{
  "success": true,
  "status": "COMPLETED",
  "resultCode": 0,
  "resultDesc": "The service request is processed successfully."
}
```

Environment Variables

Create a `.env.local` file in your project root with the following variables:

```
# Stripe Configuration
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_your_publishable_key_here
STRIPE_SECRET_KEY=sk_test_your_secret_key_here
STRIPE_WEBHOOK_SECRET=whsec_your_webhook_secret_here

# PayPal Configuration
PAYPAL_CLIENT_ID=your_paypal_client_id_here
PAYPAL_CLIENT_SECRET=your_paypal_client_secret_here
PAYPAL_API_URL=https://api-m.sandbox.paypal.com
# For production: https://api-m.paypal.com

# M-Pesa Configuration (Daraja API)
MPESA_CONSUMER_KEY=your_mpesa_consumer_key_here
MPESA_CONSUMER_SECRET=your_mpesa_consumer_secret_here
MPESA_SHORTCODE=174379
MPESA_PASSKEY=your_mpesa_passkey_here
MPESA_API_URL=https://sandbox.safaricom.co.ke
# For production: https://api.safaricom.co.ke

# Application URL
NEXT_PUBLIC_APP_URL=http://localhost:3001
# For production: https://yourdomain.com
```

Environment Variable Descriptions

Stripe

- **NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY:** Public key (safe to expose in frontend)
- **STRIPE_SECRET_KEY:** Secret key (server-side only)
- **STRIPE_WEBHOOK_SECRET:** Webhook signing secret for verifying webhook events

PayPal

- **PAYPAL_CLIENT_ID**: PayPal app client ID
- **PAYPAL_CLIENT_SECRET**: PayPal app client secret
- **PAYPAL_API_URL**: API base URL (sandbox or production)

M-Pesa

- **MPESA_CONSUMER_KEY**: Daraja API consumer key
- **MPESA_CONSUMER_SECRET**: Daraja API consumer secret
- **MPESA_SHORTCODE**: Business shortcode (Paybill or Till number)
- **MPESA_PASSKEY**: Passkey provided by Safaricom
- **MPESA_API_URL**: API base URL (sandbox or production)

Application

- **NEXT_PUBLIC_APP_URL**: Your application's base URL (used for callbacks and redirects)

Testing

Stripe Testing

1. Use Test Mode

- Ensure you're using test keys (`pk_test_` and `sk_test_`)
- Test card: 4242 4242 4242 4242
- Expiry: Any future date (e.g., 12/34)
- CVC: Any 3 digits (e.g., 123)
- ZIP: Any 5 digits (e.g., 12345)

2. Test Webhook Locally

- Use Stripe CLI: `stripe listen --forward-to localhost:3001/api/stripe-webhook`
- This forwards webhook events to your local server

3. Test Scenarios

- Successful payment
- Failed payment (use card 4000 0000 0000 0002)
- Canceled payment

PayPal Testing

1. Use Sandbox Mode

- Ensure `PAYPAL_API_URL` is set to sandbox URL
- Use sandbox test accounts from PayPal Developer Dashboard

2. Test Accounts

- PayPal provides test buyer and seller accounts
- Log in with these accounts to test payment flow

3. Test Scenarios

- Successful payment
- Canceled payment
- Payment with PayPal balance
- Payment with credit card

M-Pesa Testing

1. Use Sandbox Mode

- Ensure `MPESA_API_URL` is set to sandbox URL
- Use sandbox shortcode: 174379

2. Test Phone Numbers

- Use Safaricom test numbers if provided
- Or use your own Safaricom number in sandbox

3. Test Scenarios

- Successful STK Push
- User cancellation
- Insufficient funds
- Timeout scenarios

Troubleshooting

Common Issues

Stripe Issues

Problem: "Failed to create checkout session"

- **Solution:** Check that `STRIPE_SECRET_KEY` is set correctly
- Verify the key starts with `sk_test_` or `sk_live_`

Problem: Webhook not receiving events

- **Solution:** Verify webhook URL is publicly accessible
- Check webhook signing secret matches
- Use Stripe CLI for local testing

PayPal Issues

Problem: "Failed to get PayPal access token"

- **Solution:** Verify `PAYPAL_CLIENT_ID` and `PAYPAL_CLIENT_SECRET` are correct
- Check that `PAYPAL_API_URL` matches your environment (sandbox/live)

Problem: Order creation fails

- **Solution:** Ensure callback URLs are set correctly
- Verify the app is approved in PayPal Developer Dashboard

M-Pesa Issues

Problem: "Failed to get M-Pesa access token"

- **Solution:** Verify `MPESA_CONSUMER_KEY` and `MPESA_CONSUMER_SECRET` are correct
- Check that credentials are from the correct environment (sandbox/live)

Problem: STK Push not received

- **Solution:** Verify phone number format (should be 254XXXXXXXXXX)
- Check that callback URL is publicly accessible
- Ensure shortcode and passkey are correct

Problem: Payment status stuck on pending

- **Solution:** Check callback URL is working
- Verify callback endpoint is receiving requests
- Check Safaricom logs in Developer Portal

Debugging Tips

1. Check Server Logs

- Monitor console logs for API errors
- Check network requests in browser DevTools

2. Verify Environment Variables

- Ensure all required variables are set
- Check for typos in variable names
- Verify values match your account credentials

3. Test API Routes Directly

- Use tools like Postman or curl to test API routes
- Verify request/response formats

4. Check Payment Gateway Dashboards

- Stripe Dashboard: View payments and webhook events
- PayPal Dashboard: View orders and transactions
- M-Pesa Portal: View API logs and transactions

Security Considerations

1. **Never expose secret keys** in client-side code
2. **Use HTTPS** in production for all API calls
3. **Validate webhook signatures** (Stripe and M-Pesa)
4. **Sanitize user inputs** before processing
5. **Implement rate limiting** for API routes
6. **Use environment variables** for all sensitive data
7. **Keep dependencies updated** for security patches

Support and Resources

Stripe

- Documentation: <https://stripe.com/docs>
- Support: <https://support.stripe.com>
- Status: <https://status.stripe.com>

PayPal

- Documentation: <https://developer.paypal.com/docs>
- Support: <https://www.paypal.com/support>
- Status: <https://www.paypal-status.com>

M-Pesa (Daraja API)

- Documentation: <https://developer.safaricom.co.ke/docs>
- Support: developer@safaricom.co.ke
- Portal: <https://developer.safaricom.co.ke>

Changelog

Version 2.0 (Current)

-  Improved UI with dynamic animated background
-  Better payment method selection with visual cards
-  Fixed donation amount calculation accuracy
-  M-Pesa now only requires phone number
-  Improved form layout and spacing
-  Better error handling and user feedback
-  Responsive design improvements

Version 1.0

- Initial implementation with basic payment integrations
- Multi-currency support
- Basic form layout

License

This donation system is part of the Workers Rights Watch website project.

Last Updated: January 2025